

Management CLI Automations

Objectives

1. Connect to the R82.10 lab SMS server and verify the API is reachable.
2. Identify the four entry points into the Management API surface.
3. Read an unfamiliar mgmt_cli flow and decide whether to run it.
4. Apply the mental model: session → do something → publish → install-policy.
5. Write a customer-ready rollback plan for an API-driven change.

Lab Environment

Resource	Address	Credentials
SMS	10.1.1.100	admin / Cpwins!1
GW	10.1.1.111	admin / Cpwins!1
Jump server (Windows)	via MobaXterm bookmark	—

The lab is **pre-configured** by the instructor. The Management API listens on all IPs; the Standard policy package exists with GW as the target.

Pre-Flight

1. SSH from **MobaXterm** to **SMS** (10.1.1.100).
2. Run **api status** and confirm:
 - Overall, API Status: Started
 - Accessibility: All IP addresses

```

Terminal Sessions View X server Tools Settings Macros Help
Session Servers Tools Sessions View Split MultiExec Tunneling Packages Settings Help Zoom out Zoom in
Quick connect...
User sessions
  > RDP
  > SSH
    10.1.1.100 (SMS)
    10.1.1.111 (GW)
    10.1.3.33 (AI-Ubuntu)
    203.0.113.5 (Kali-Linux)

[Expert@SMS:0]# api status

API Settings:
-----
Accessibility:          Require all granted
Automatic Start:       Enabled

Processes:
-----
Name      State    PID      More Information
-----
API       Started  12499
CPM       Started  12499    Check Point Security Management Server is running and ready
FWM       Started  11887
APACHE    Started  10568

Port Details:
-----
JETTY Internal Port:    65424
JETTY Documentation Internal Port: 55775
APACHE Gaia Port:      443

Profile:
-----
Machine profile:        Large env resources profile with SME or Dedicated Log Server
CPM heap size:          2304m

Apache port retrieved from: dbget http:ssl_port

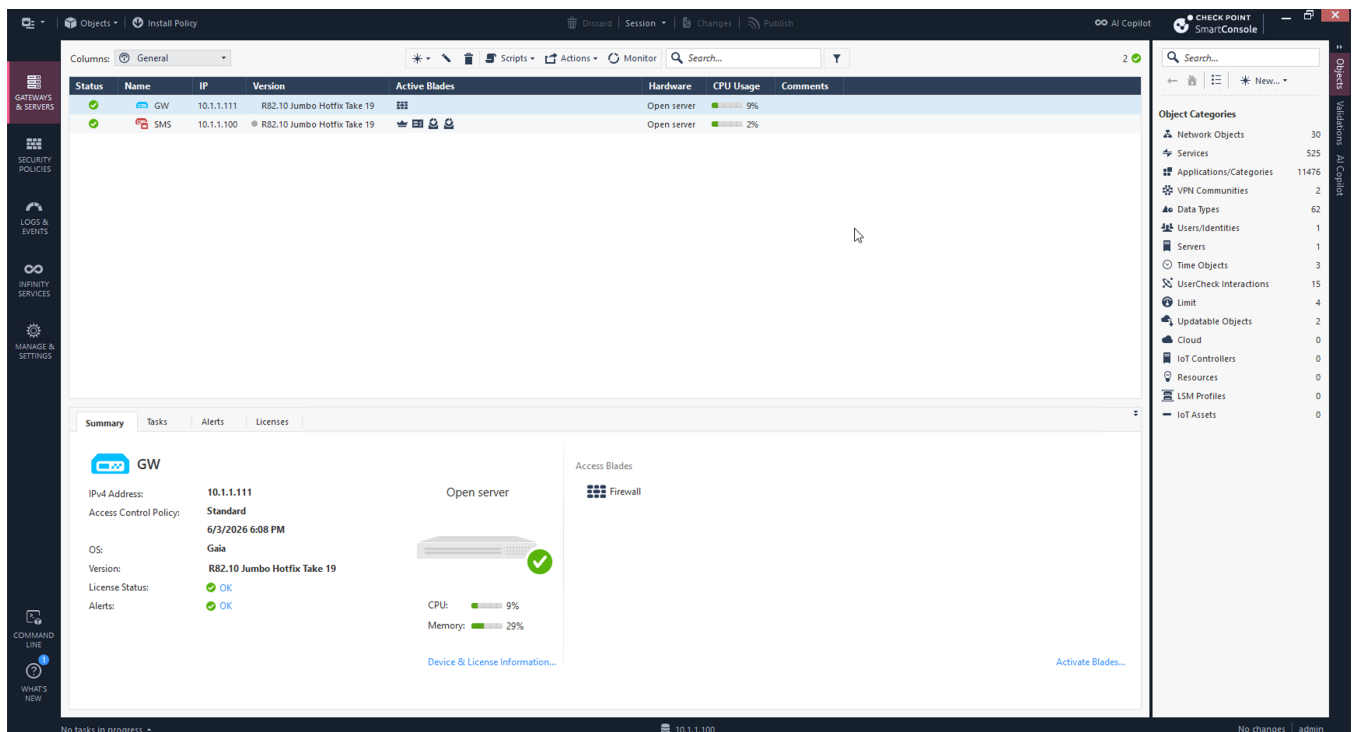
-----
Overall API Status: Started
-----

API readiness test SUCCESSFUL. The server is up and ready to receive connections

Notes:
-----
To collect troubleshooting data, please run 'api status -s <comment>'

```

3. Open **SmartConsole** on the jump server; connect to 10.1.1.100, log in as admin / Cpwins!1.



If API status shows anything different, raise your hand.

Task 1: The Four Entry Points (10 Minutes)

Run each of these once. **Don't memorize the syntax — just make sure all four hit the same API.**

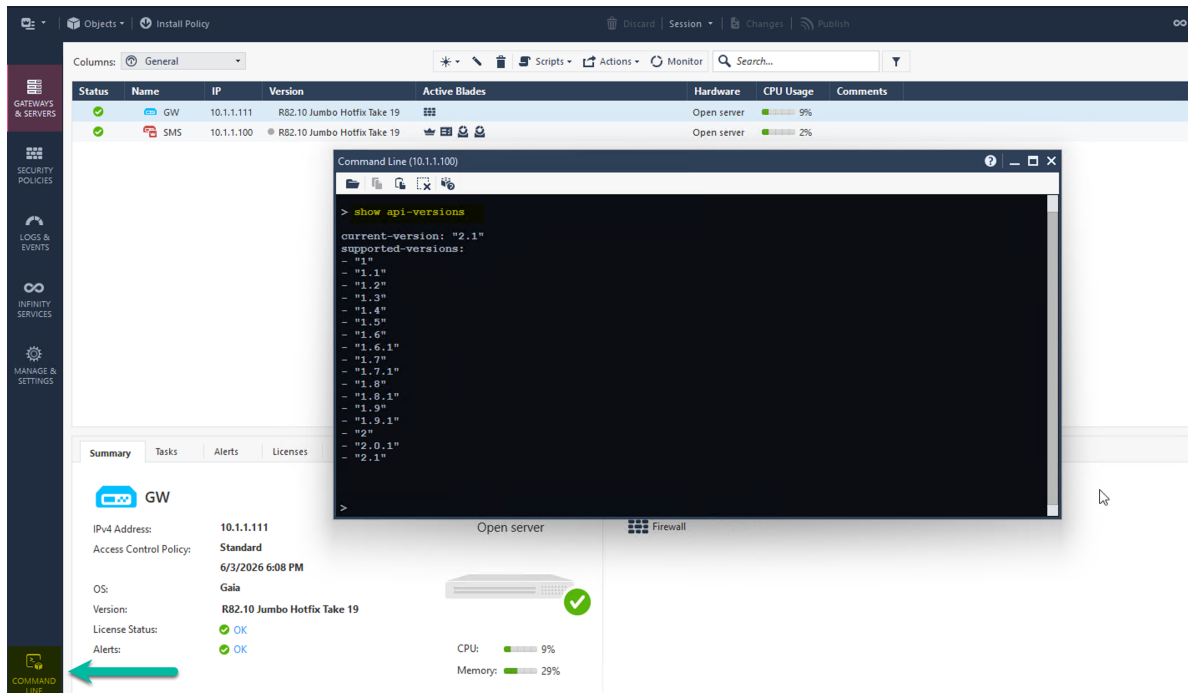
1. In your SMS SSH session, type the following command to see the supported API versions:

```
mgmt_cli -r true show api-versions
```

```
[Expert@SMS:0]# mgmt_cli -r true show api-versions
current-version: "2.1"
supported-versions:
- "1"
- "1.1"
- "1.2"
- "1.3"
- "1.4"
- "1.5"
- "1.6"
- "1.6.1"
- "1.7"
- "1.7.1"
- "1.8"
- "1.8.1"
- "1.9"
- "1.9.1"
- "2"
- "2.0.1"
- "2.1"
```

2. In SmartConsole, click the **Command Line** icon (left toolbar). Type:

```
show api-versions
```



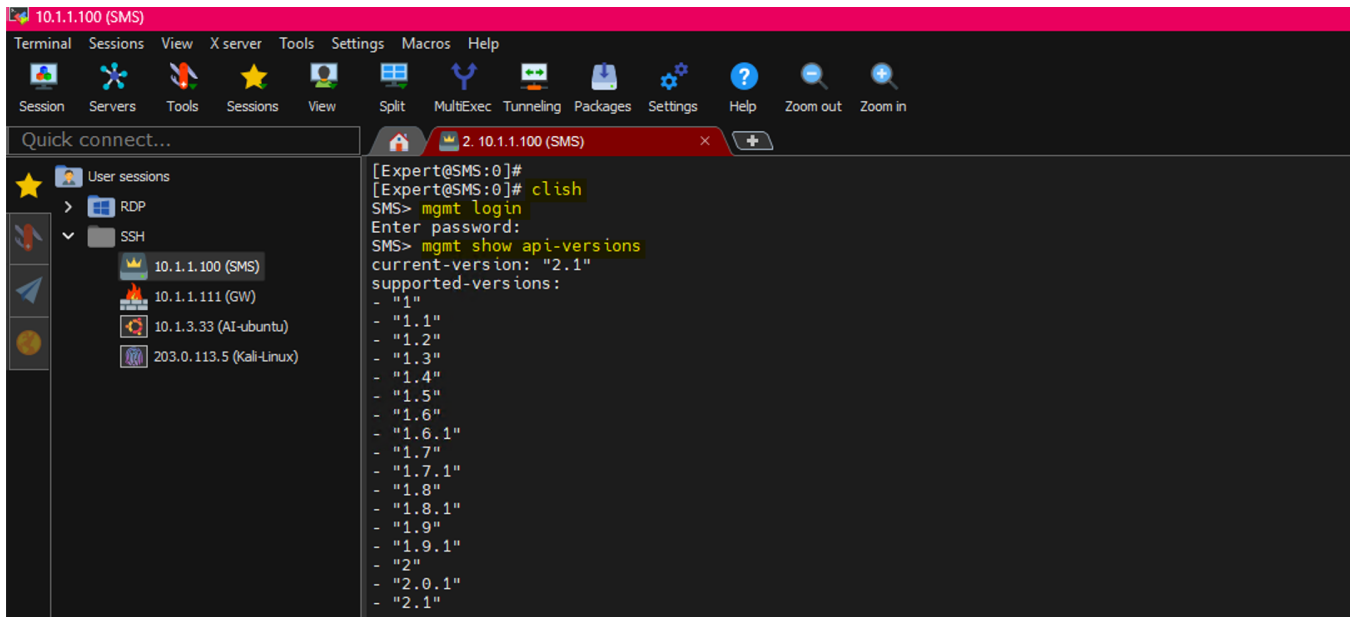
The screenshot shows the Check Point SmartConsole interface. The left sidebar contains icons for 'GATEWAYS & SERVICES', 'SECURITY POLICIES', 'LOGS & EVENTS', 'INFINITY SERVICES', 'MANAGE & SETTINGS', and 'COMMAND LINE'. The 'COMMAND LINE' icon is highlighted with a green arrow. The main window displays a table of objects (GW and SMS) and a 'Command Line' window showing the output of the 'show api-versions' command.

Status	Name	IP	Version	Active Blades	Hardware	CPU Usage	Comments
✓	GW	10.1.1.111	R82.10 Jumbo Hotfix Take 19	...	Open server	9%	
✓	SMS	10.1.1.100	R82.10 Jumbo Hotfix Take 19	...	Open server	2%	

```
Command Line (10.1.1.100)
> show api-versions
current-version: "2.1"
supported-versions:
- "1"
- "1.1"
- "1.2"
- "1.3"
- "1.4"
- "1.5"
- "1.6"
- "1.6.1"
- "1.7"
- "1.7.1"
- "1.8"
- "1.8.1"
- "1.9"
- "1.9.1"
- "2"
- "2.0.1"
- "2.1"
```

3. Back in the SSH session, enter clish and run:

```
mgmt login (use the admin password "Cpwins!1")  
mgmt show api-versions
```

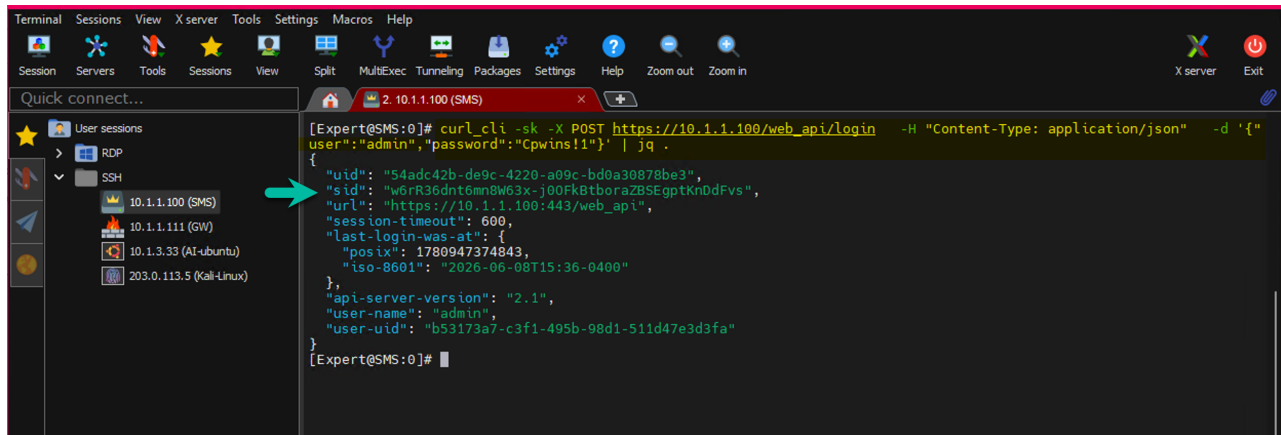


The screenshot shows a terminal window titled "10.1.1.100 (SMS)". The left sidebar shows a tree view with "SSH" selected, and "10.1.1.100 (SMS)" is the active session. The terminal output is as follows:

```
[Expert@SMS:0]#  
[Expert@SMS:0]# clish  
SMS> mgmt login  
Enter password:  
SMS> mgmt show api-versions  
current-version: "2.1"  
supported-versions:  
- "1"  
- "1.1"  
- "1.2"  
- "1.3"  
- "1.4"  
- "1.5"  
- "1.6"  
- "1.6.1"  
- "1.7"  
- "1.7.1"  
- "1.8"  
- "1.8.1"  
- "1.9"  
- "1.9.1"  
- "2"  
- "2.0.1"  
- "2.1"
```

4. REST /web_api/ directly. From your SSH session on the SMS, run the following command from Expert mode:

```
curl_cli -sk -X POST https://10.1.1.100/web_api/login \  
-H "Content-Type: application/json" \  
-d '{"user":"admin","password":"Cpwins!1"}' | jq .
```



The screenshot shows the same terminal window. A red arrow points to the command being executed. The output is a JSON object:

```
[Expert@SMS:0]# curl_cli -sk -X POST https://10.1.1.100/web_api/login -H "Content-Type: application/json" -d '{"user":"admin","password":"Cpwins!1"}' | jq .  
{  
  "uid": "54adc42b-de9c-4220-a09e-bd0a30878be3",  
  "sid": "w6rR36dnt6mn8W63x-j00Fk8tborazBSEgptKnDdFvs",  
  "url": "https://10.1.1.100:443/web_api",  
  "session-timeout": 600,  
  "last-login-was-at": {  
    "posix": 1780947374843,  
    "iso-8601": "2026-06-08T15:36-0400"  
  },  
  "api-server-version": "2.1",  
  "user-name": "admin",  
  "user-uid": "b53173a7-c3f1-495b-98d1-511d47e3d3fa"  
}  
[Expert@SMS:0]#
```

You get a JSON response with a sid. **Save the sid for now** — we'll come back to sessions in Task 3. What you just learned

The SMS API has **one surface, four front doors**. Choose the front door that fits the context:

Context	Front door
Inside SmartConsole, quick poke	SmartConsole CLI
Scripts on the SMS server	mgmt_cli
Already in clish on Gaia	mgmt ...
External system / pipeline / AI agent	REST /web_api/



Detailed steps with examples can be found at the API Portal

https://sc1.checkpoint.com/documents/latest/api_reference/index.html

Task 2: Trace a Session-less Flow (15 Minutes)

The block below adds three hosts and installs the policy using session-less calls. Each line is a self-contained mgmt_cli transaction. Read it carefully before running it.

```
mgmt_cli -r true add host name web1 ip-address 10.20.30.41
mgmt_cli -r true add host name web2 ip-address 10.20.30.42
mgmt_cli -r true add host name web3 ip-address 10.20.30.43
mgmt_cli -r true install-policy access true policy-package "Standard"
targets.1 "GW"
```

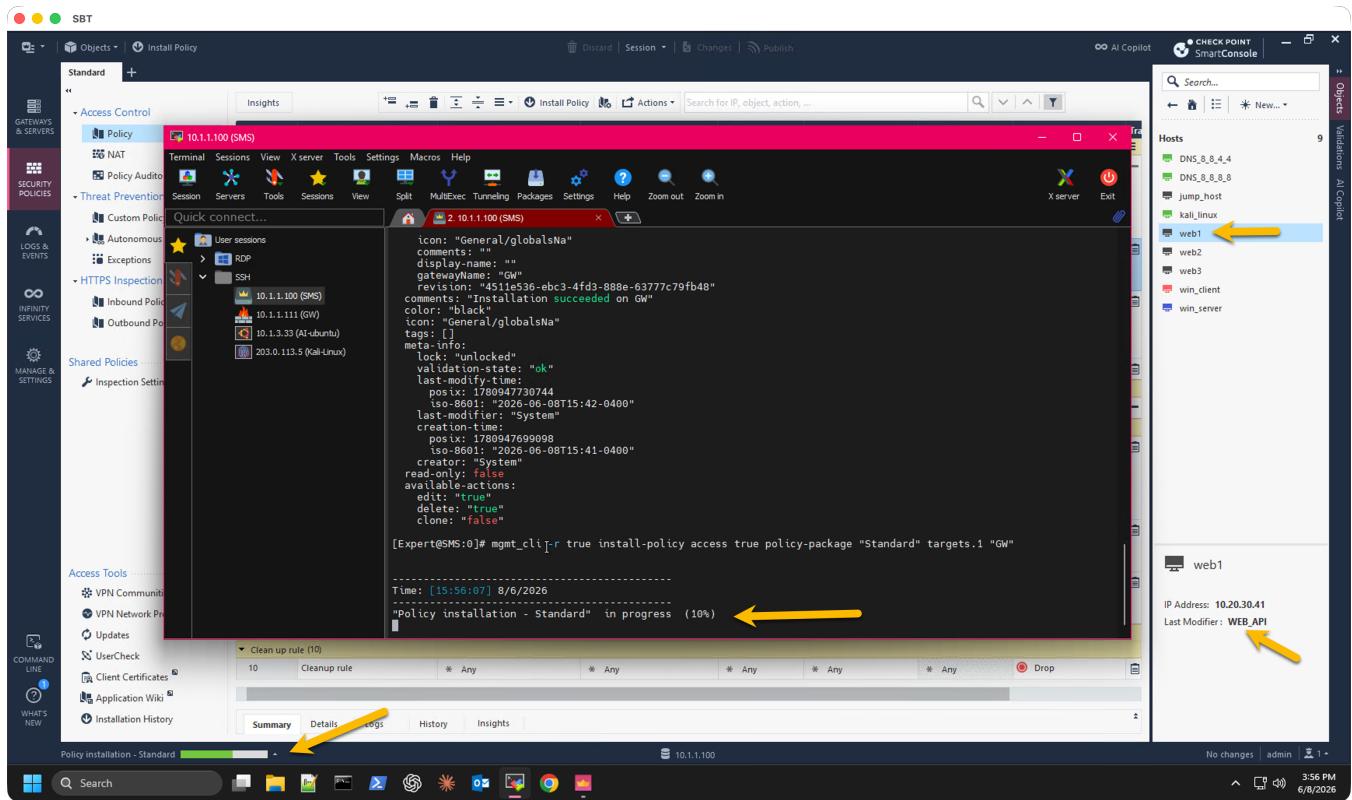
2a — Before you run it

1. Each of the four lines runs as its own independent API session. What does that mean for what is visible in SmartConsole between lines?
2. If line 4 (install-policy) fails for any reason, how would you undo the three host adds as a single unit?
3. Which of these four lines is safe to re-run on retry, and which are not?

2b — Run it

Run all four lines in order. Open SmartConsole and confirm:

1. The three host objects appear under **Network Objects** → **Hosts**.
2. A policy install task appears under **Logs & Monitor** → **Open Tasks**.



2c — The lesson

Without an explicit session, **each** `mgmt_cli` call opens its own session and publishes when it closes. The four lines above are **four independent transactions, not one**. Convenient for ad-hoc reads. Dangerous for multi-step writes — there is no atomic rollback. If line 4 fails, the host objects are already published and visible.

Task 3: Session-Based Flow (20 Minutes)

This is the production pattern. One session wraps the writes; you publish once at the end. The mechanic to know: `mgmt_cli -s` expects a **session file path**, not the SID string. Log in once and redirect the whole login output to a file with `> /tmp/sid.txt`. Then pass that file to every subsequent call with `-s /tmp/sid.txt`. Run the block below on the SMS shell. (Note: the password `Cpwins!1` contains `!`, which bash treats as history expansion. Escape it as `Cpwins\!1` or single-quote the password.)

```
# 1. Log in and save the whole login output to a session file
mgmt_cli login -u admin -p Cpwins\!1 > /tmp/sid.txt
```

```
# 2. Make all changes against the session file (NOT yet visible to other users)
mgmt_cli add host name web4 ip-address 10.20.30.44 -s /tmp/sid.txt
mgmt_cli add host name web5 ip-address 10.20.30.45 -s /tmp/sid.txt
mgmt_cli add host name web6 ip-address 10.20.30.46 -s /tmp/sid.txt

# 3. Publish – the single commit point
mgmt_cli publish -s /tmp/sid.txt

# 4. Install the policy in the same session
mgmt_cli install-policy access true policy-package "Standard" targets.1 "GW" -s /tmp/sid.txt

# 5. Close the session
mgmt_cli logout -s /tmp/sid.txt
```

3a — Questions before you run it

- Between steps 2 and 3, are web4 / web5 / web6 visible to other SmartConsole users? Why or why not?
- If step 2's second line (web5) fails, what is the clean way to abandon the change without publishing? (Hint: look up `mgmt_cli discard`.)
- Why does logout matter? What happens to the session if you skip it?

3b — The lesson

A session is your **transaction boundary**. You batch writes into one logical change, validate, publish atomically, and clean up. This is the only sane pattern for scripts that mutate customer state.

Task 4: Find the Bug (10 Minutes)

The script below was intended to add three hosts and install the policy as a single atomic change. It contains **at least two bugs** that prevent it from doing that.

```
mgmt_cli login -u admin -p Cpwins\!1 > /tmp/sid.txt

mgmt_cli add host name web7 ip-address 10.20.30.47 -s /tmp/sid.txt
mgmt_cli add host name web8 ip-address 10.20.30.48 -s /tmp/sid.txt
mgmt_cli add host name web9 ip-address 10.20.30.49

mgmt_cli install-policy access true policy-package "Standard" targets.1 "GW" -s /tmp/sid.txt
mgmt_cli logout -s /tmp/sid.txt
```

Questions

1. Read the script carefully. Find at least two bugs **without running it**. Write them down.
 2. Trace the script line by line and predict what will actually happen if you run it as-is.
 3. Fix the script and run it. Verify the result in SmartConsole.
- (Solutions are in Appendix A — do not peek until you have tried.)*

Task 5: Pick the Right Pattern (5 Minutes)

Decision shortcut for production scripts:

One-off read (show ...) from the SMS shell	Session-less (-r true)
Quick interactive single-line write	Session-less — aware it auto-publishes
Any multi-step write that must succeed or roll back as a unit	Session-based
Anything in CI/CD, Ansible, Terraform, scheduled jobs, or customer scripts	Session-based, always

Cleanup

Remove every host you created during the lab so the next cohort starts clean:

```
mgmt_cli login -u admin -p Cpwins\!1 > /tmp/sid.txt

for host in web1 web2 web3 web4 web5 web6 web7 web8 web9; do
    mgmt_cli delete host name "$host" -s /tmp/sid.txt
done

mgmt_cli publish -s /tmp/sid.txt
mgmt_cli logout -s /tmp/sid.txt
```

Appendix A — Task 4 Solutions

Bug 1 — missing session flag on line 4. The line `mgmt_cli add host name web9 ip-address 10.20.30.49` is missing `-s /tmp/sid.txt`. It runs in its own session, auto-publishes, and web9 appears in the policy database independently of the intended atomic change.

Bug 2 — no publish before install-policy. web7 and web8 exist only inside the session. install-policy operates against the published database, so it would install a policy that does not include those hosts. The session must be published first.

Fixed script:

```
mgmt_cli login -u admin -p Cpwins\!1 > /tmp/sid.txt

mgmt_cli add host name web7 ip-address 10.20.30.47 -s /tmp/sid.txt
mgmt_cli add host name web8 ip-address 10.20.30.48 -s /tmp/sid.txt
mgmt_cli add host name web9 ip-address 10.20.30.49 -s /tmp/sid.txt

mgmt_cli publish -s /tmp/sid.txt
mgmt_cli install-policy access true policy-package "Standard" targets.1 "GW" -
s /tmp/sid.txt
mgmt_cli logout -s /tmp/sid.txt
```